

EXPERIMENT-9

Name : Sharon Elango

Batch : 3

Roll No : 62

Aim : To implement side effects in React using the useEffect Hook and fetch data from a public API using Fetch API and async/await, and dynamically display the data as a list/cards.

#Code for [App.js](#):

```
import React from 'react';
import UserList from './UserList';
import './App.css';
```

```
function App() {
  return (
    <div className="App">
      <h1>User Directory</h1>
      <UserList />
    </div>
  );
}
```

```
export default App;
```

#Code for [UserList.js](#):

```
import React, { useState, useEffect } from 'react';
```

```
const UserList = () => {
  // Step 4: Initialize State [cite: 91, 98]
  const [users, setUsers] = useState([]);    // Data as empty array [cite: 100]
  const [loading, setLoading] = useState(true); // Loading as true [cite: 101]
  const [error, setError] = useState(null);  // Error as null [cite: 102]
```

```

// Step 5: Implement useEffect Hook [cite: 103]
useEffect(() => {
  // Create an async function inside useEffect [cite: 106]
  const fetchUsers = async () => {
    try {
      // Step 5.2: Call the API using Fetch and await response [cite: 108,
109]
      const response = await
fetch('https://jsonplaceholder.typicode.com/users');

      if (!response.ok) {
        throw new Error('Failed to fetch data');
      }

      // Step 5.3: Convert response to JSON [cite: 113]
      const data = await response.json();

      // Step 5.4: Update state with received data [cite: 114]
      setUsers(data);
    } catch (err) {
      // Step 7: Handle Errors [cite: 121, 122]
      setError(err.message);
    } finally {
      // Step 5.5: Set loading to false [cite: 112]
      setLoading(false);
    }
  };

  fetchUsers();
}, []); // Empty dependency array ensures it runs only once on mount [cite:
54, 116]

// Step 6: Handle Loading State [cite: 117, 119]
if (loading) return <div className="loader">Loading users...</div>;

```

```

// Step 7: Display error message in UI if it exists [cite: 122]
if (error) return <div className="error-msg">Error: {error}</div>;

// Step 8: Dynamically Display Data using map() [cite: 123, 124]
return (
  <div className="user-grid">
    {users.map((user) => (
      <div key={user.id} className="user-card">
        <h3>{user.name}</h3> { /* [cite: 130] */}
        <p><strong>Email:</strong> {user.email}</p> { /* [cite: 132] */}
        <p><strong>Company:</strong> {user.company.name}</p> { /* [cite:
133] */}
        <p><strong>Phone:</strong> {user.phone}</p> { /* [cite: 134] */}
      </div>
    ))}
  </div>
);
};

export default UserList;

```

#Code for App.css:

```

.App {
  text-align: center;
  padding: 20px;
  font-family: Arial, sans-serif;
}

.user-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
  gap: 20px;
  padding: 20px;
}

```

```

}

.user-card {
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 15px;
  box-shadow: 2px 4px 8px rgba(0,0,0,0.1);
  background-color: #fff;
  text-align: left;
}

.user-card h3 {
  margin-top: 0;
  color: #007bff;
}

.loader {
  font-size: 1.5rem;
  margin-top: 50px;
}

.error-msg {
  color: red;
  font-weight: bold;
}

```

#Code for App.jsx:

```

import React from 'react';
import UserList from './UserList';
import './App.css';

```

```
/**
```

```
 * App Component
```

```
 * Step 3: Create a Functional Component and render UserList inside App.
```

```

*/
function App() {
  return (
    <div className="App">
      <header className="app-header">
        <h1>React API Fetch Experiment</h1>
        <p>AIML Department - Second Year [cite: 3, 4]</p>
      </header>

      <main>
        {/* Rendering the component that handles side effects and API calls
[cite: 5] */}
        <UserList />
      </main>

      <footer className="app-footer">
        <p>Experiment 9: React Side Effects & API Calls [cite: 5]</p>
      </footer>
    </div>
  );
}

export default App;

```

#Output :

<>🔄🌐localhost:3000

🔍📧🏠👤➤🔖📄☰👤🏠🧠AI

User Directory

Leanne Graham Email: Sincere@april.biz Company: Romaguera-Crona Phone: 1-770-736-8031 x56442	Ervin Howell Email: Shanna@melissa.tv Company: Deckow-Crist Phone: 010-692-6593 x09125	Clementine Bauch Email: Nathan@yesenia.net Company: Romaguera-Jacobson Phone: 1-463-123-4447	Patricia Lebsack Email: Julianne.OConner@kory.org Company: Robel-Corkery Phone: 493-170-9623 x156	Chelsey Dietrich Email: Lucio_Hettinger@annie.ca Company: Keebler LLC Phone: (254)954-1289
Mrs. Dennis Schulist Email: Karley_Dach@jasper.info Company: Considine-Lockman Phone: 1-477-935-8478 x6430	Kurtis Weissnat Email: Telly.Hoeger@billy.biz Company: Johns Group Phone: 210.067.6132	Nicholas Runolfsdottir V Email: Sherwood@rosamond.me Company: Abernathy Group Phone: 586.493.6943 x140	Glenna Reichert Email: Chaim_McDermott@dana.io Company: Yost and Sons Phone: (775)976-6794 x41206	Clementina DuBuque Email: Rey.Padberg@karina.biz Company: Hoeger LLC Phone: 024-648-3804